Data-Mart-Erstellung in SQL (DLBDSPBDM01_D): Kurze Zusammenfassung der Implementierung

Miskovic, Dominik · Matrikelnummer: 14133980 · Kurs: DLBDSPBDM01_D

Erarbeitungs-/Reflexionsphase

Das Datenbankschema der Buchtausch-App wurde in PostgreSQL 16 umgesetzt und besteht aus 14 Tabellen: den 13 Entitäten aus dem ER-Modell der Konzeptionsphase sowie der Verbindungstabelle buch_autor zur Auflösung der N:M-Beziehung zwischen Buch und Autor:in. Beide Ternärbeziehungen aus Phase 1 (AUSLEIHT und ANGEBOTEN_ALS) wurden als eigene Tabellen mit Surrogatschlüssel umgesetzt (ausleihvorgang, angebot). Das ist eine Standardtechnik, wenn eine Beziehung selbst wieder referenziert werden muss, wie es das Tutor-Feedback zur Bewertung erforderte.

Alle drei Rückmeldungen aus dem Feedback zu Phase 1 wurden eingearbeitet. Erstens ist die Bewertung jetzt über einen UNIQUE-Fremdschlüssel an einen konkreten Ausleihvorgang gekoppelt statt direkt an Buch und Benutzer:in, zusätzlich per Trigger abgesichert, dass nur abgeschlossene ('zurueckgegeben') Ausleihvorgänge bewertet werden können. Zweitens wurde die Beziehung zwischen Benutzer und Adresse von 1:1 auf 1:N geändert, sodass mehrere Adressen je Person möglich sind, ergänzt um ein Flag für die Hauptadresse. Drittens wurden die geforderten Constraints konsequent implementiert: CHECK-Constraints auf die Sternebewertung (1 bis 5), auf positive Leihdauer und auf die zeitliche Reihenfolge von Ausleih- und Zeitfenster-Daten, UNIQUE auf die E-Mail-Adresse sowie ein zusätzlicher Trigger, der verhindert, dass Nutzende ihr eigenes Exemplar ausleihen.

Die Datenbank wurde mit insgesamt über 175 Dummy-Datensätzen befüllt (mindestens 10 pro Tabelle), alle referenziell konsistent und alle Geschäftsregeln respektierend. Die räumliche Suchanforderung ('Bücher in meiner Nähe') wurde über eine Haversine-Distanzformel direkt in SQL umgesetzt, ganz ohne zusätzliche Erweiterung wie PostGIS, und liefert nachweislich plausible, nach Entfernung sortierte Ergebnisse. Zwei Negativtests belegen, dass die implementierten Trigger tatsächlich greifen und fehlerhafte Eingaben zuverlässig zurückweisen: eine versuchte Selbst-Ausleihe sowie eine Bewertung für einen noch nicht abgeschlossenen Ausleihvorgang wurden beide korrekt abgelehnt.

Alle 15 dokumentierten Testfälle (ein SELECT-Statement je Entität, plus die räumliche Suche und die zwei Negativtests) wurden gegen die tatsächlich befüllte Datenbank ausgeführt; die zugehörigen Screenshots in der Präsentation zeigen die echten Ergebnisse aus psql.